TalentoSaaS — Gestão de Talentos e Folha (RH)

Atividade Prática 04 · Entrega 1 (19/10/2026): Arquitetura, fluxo da aplicação e distribuição das funções · **Grupo B · SaaS de RH**

Integrantes:
Adriano Ferreira · Fabricio Carmo
Lucas Soares · Nefi Moraes

Problema de negócio. O RH recebe um currículo para uma vaga e precisa decidir rápido: o candidato atende aos requisitos, quanto ele vai receber e custar, e o que precisa estudar para fechar as lacunas. O TalentoSaaS faz a triagem ATS (AWS) e o cálculo de salário líquido e encargos (Azure) **em paralelo** (`Promise.allSettled`); em seguida, o score e as lacunas do ATS alimentam, **em cascata**, o recomendador de trilha de capacitação (GCP).

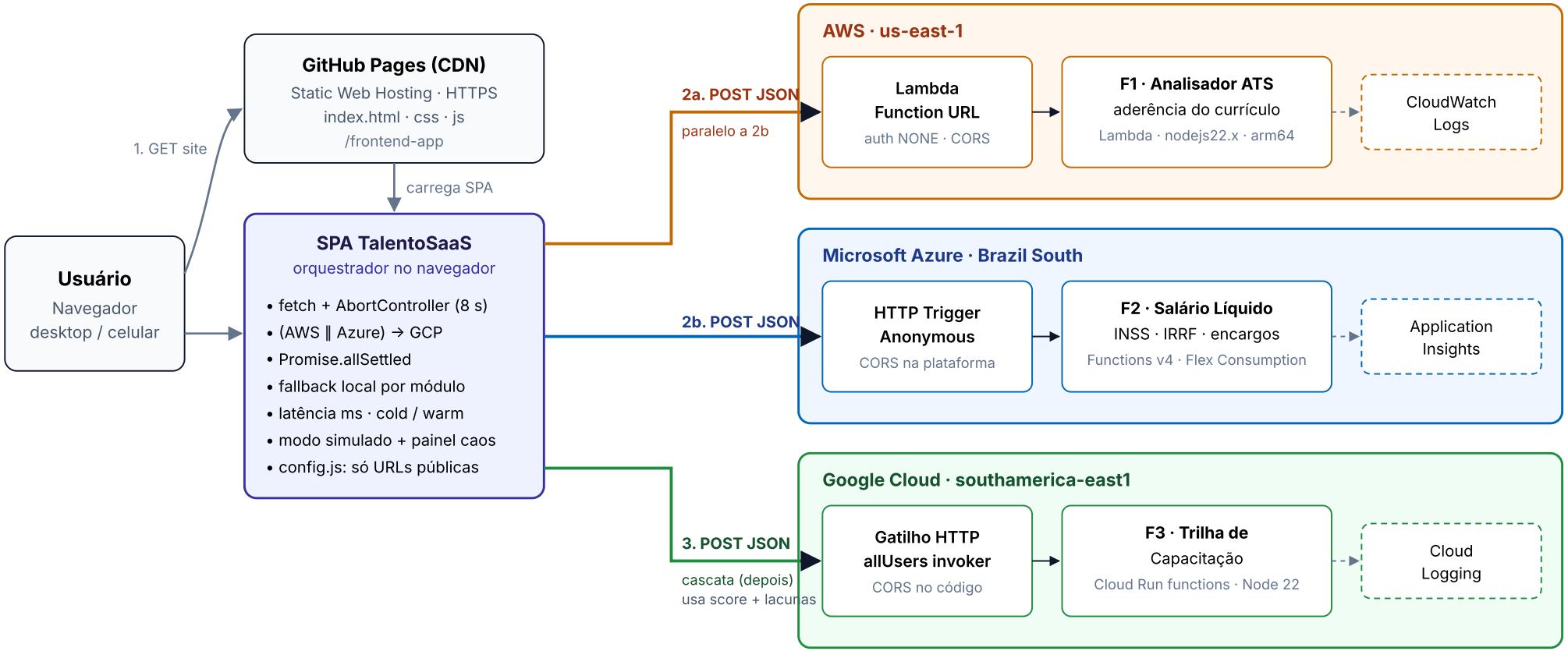


Figura 1 — Arquitetura da solução. Usuário → Hospedagem do Frontend → Requisições HTTPS/JSON → AWS / Azure / GCP. Nenhuma credencial fica no frontend: os três endpoints são públicos e o SPA guarda apenas as URLs. Linhas tracejadas = logs enviados ao monitoramento de cada nuvem (usado na "Auditoria nos Consoles").

Fluxo da aplicação (orquestração paralelo + cascata, com degradação graciosa)

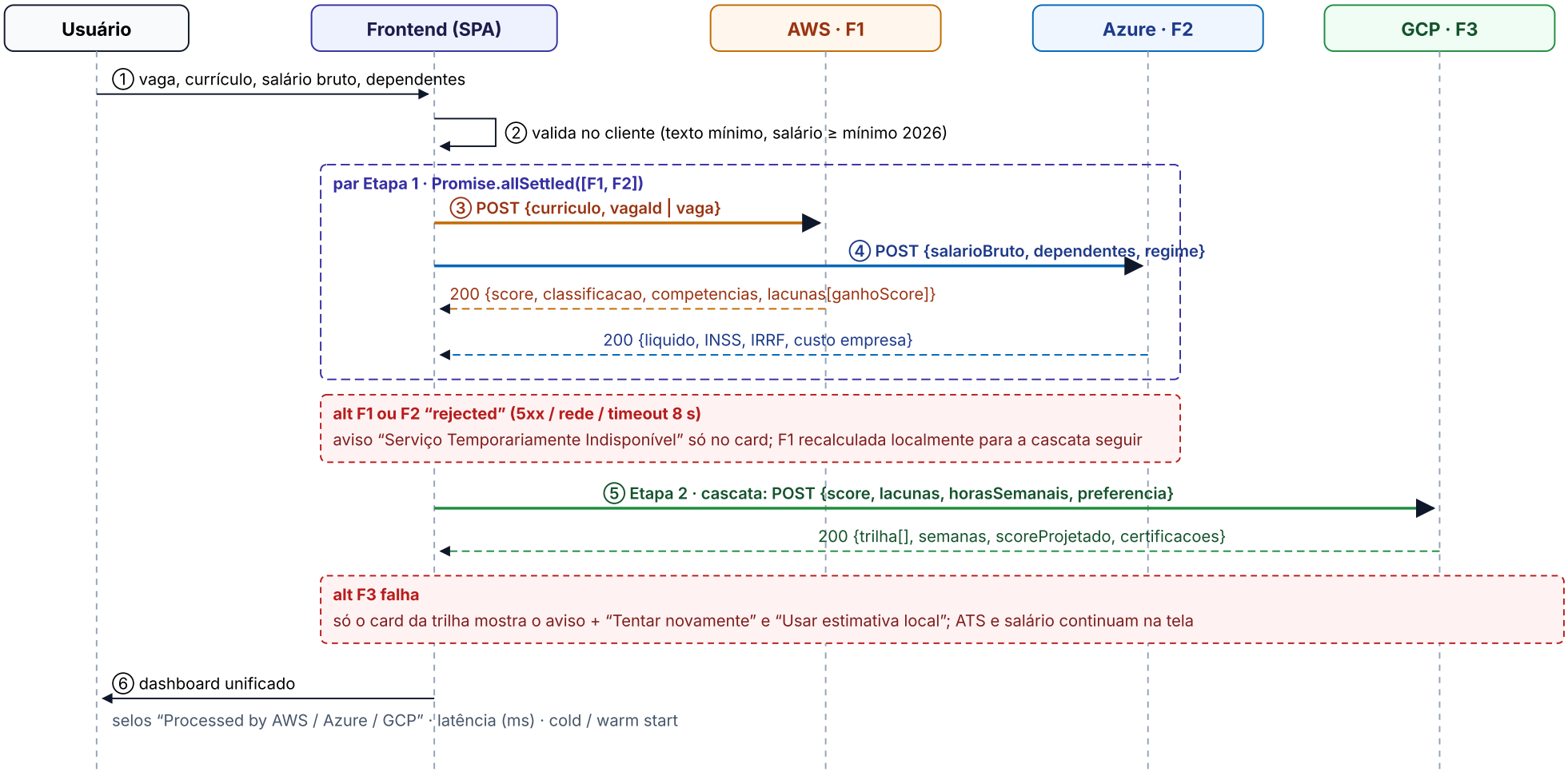


Figura 2 — Diagrama de sequência. Etapa 1: ATS (AWS) e salário (Azure) não dependem um do outro, então rodam em paralelo. Etapa 2: a trilha (GCP) precisa do score e das lacunas do ATS, por isso vem em cascata. F1 tem fallback local completo para a cascata nunca travar.

Qual função roda em cada plataforma

Plataforma	Função	Entrada (Request JSON)	Saída (Response JSON)	Serviço e configuração
AWS Lambda · us-east-1	**F1 — Analisador de Aderência de Currículo (ATS)** O candidato atende à vaga?	`curriculo` (texto), `vagaId` (5 vagas prontas) ou `vaga` {obrigatorias, desejaveis, experienciaMinAnos}	**score 0–100%**, classificação, recomendação, competências encontradas (com nº de menções), **lacunas** com ganho de score, anos de experiência, diferenciais	Node.js 22 (arm64, 256 MB, 15 s) · **Lambda Function URL** auth NONE · CORS na URL · permissões `InvokeFunctionUrl` + `InvokeFunction` · CloudWatch (sem gravar o texto do currículo — LGPD)
Azure Functions · Brazil South	**F2 — Calculadora de Salário Líquido e Encargos** Quanto recebe e quanto custa?	`salarioBruto`, `dependentes`, `regime` (lucro presumido / simples), `valeTransporte`	INSS progressivo 2026 faixa a faixa, IRRF 2026 com **redução da Lei 15.270/2025** e desconto simplificado, VT 6%, **líquido**, alíquota efetiva, custo da empresa (FGTS, CPP, RAT, terceiros, 13º, férias + 1/3)	Functions v4, Node 22, plano **Flex Consumption** · **HTTP Trigger anonymous** · rota `/api/salario-liquido` · CORS na plataforma · Application Insights
GCP Cloud Run functions · southamerica-east1	**F3 — Recomendador de Trilha de Capacitação** O que estudar para fechar as lacunas?	`score` e `lacunas` (vindos da F1), `horasSemanais`, `preferencia` (gratuito / qualquer)	trilha ordenada (obrigatórias primeiro) com cursos e certificações, plataforma, horas, **semana de início/fim**, data prevista de conclusão e **score projetado**	Node 22, gatilho HTTP, **invocações não autenticadas** · CORS tratado no código (`ALLOWED_ORIGINS`) · Cloud Logging

Padrões comuns às três funções

- `Content-Type: application/json` na entrada e na saída.
- **200** sucesso · **400** validação (lista `detalhes`) · **500** falha interna · **405** método.
- `GET` = health check (aquece a função e testa o CORS).
- Toda resposta traz `meta`: provedor, região, `coldStart`, `durationMs`, `requestId`.
- `?chaos=erro` força 500 e `?chaos=lento` passa do timeout — usados no teste de resiliência ao vivo.
- Lógica de negócio em módulo puro, testada com `node --test`.

Por que esse nível é moderado/difícil

- **Paralelo + cascata** no mesmo fluxo, com fallback que preserva a cascata.
- ATS com normalização de acentos, dicionário de sinônimos (ex.: k8s = Kubernetes) e pesos.
- Experiência estimada pelos períodos do currículo, sem contar sobreposição.
- Legislação real de 2026: INSS progressivo, IRRF com redutor da Lei 15.270/2025.
- CORS configurado de **três formas diferentes** (URL, plataforma, código).
- Métricas de cold/warm start registradas e exportáveis em CSV.

Cronograma

Data	Entrega	Conteúdo
19/10/2026	1. Planejamento e desenho	Este documento: tema, arquitetura, fluxo e distribuição das funções.
26/10/2026	2. Frontend publicado	SPA no GitHub Pages em **modo simulado** (mesma lógica das funções no navegador, latência e cold start simulados, painel de caos).
16/11/2026	3. Projeto final + apresentação	Deploy nas 3 nuvens, troca para modo “Nuvem”, medições cold/warm, relatório PDF, README e demonstração com queda de um provedor.